MANAGEMENT BRIEFING

From spec-driven to intent-driven development

Why SDD repeats the problem of upfront design specs, and what owning the intent changes.

Based on K. V. Ahuja, "The Method That Replaces Spec-Driven Development — IDSD" (May 2026) and U. Barbini, "Spec-Driven Development: The New Waterfall" (August 2026)

The problem with spec-driven development

One document, four jobs

The spec has to be intent, definition of done, workflow and technical context at once, written however the author felt that day.

A complete spec cannot be written up front

OpenAI's Symphony spec (2,169 lines) was distilled from a system that already ran. It was written last, not first.

The agent fills every hole

Gaps become agent decisions, at volume. Teams feel fast while being wrong; a 2025 METR trial found developers slower with AI but sure they were faster.

The cost does not disappear. Rework and wasted tokens move down the line to the client's invoice.

Source: Ahuja (May 2026). His own example: three days of rework, about $985 in tokens, from one good spec left unattended.

A second view: SDD as the new waterfall

SAME PROMISE EACH TIME: ARCHITECTS DESCRIBE INTENT, CODE FOLLOWS

CASE tools
late 1980s

→

RUP, CMMI
heavy process

→

Model Driven Architecture
model generates code

→

SDD
spec generates code (2025+)

The spec becomes the system

A spec precise enough to generate a system is as complex as the system. A vague one makes the agent guess, and drift starts early.

Regenerating is rewriting

The same spec yields different systems. Fixes made since the last run are lost, so teams edit code and the spec quietly goes stale.

Document churning

Agents produce plan after plan that reads well but was never tested against reality. Only running code has to match reality.

Where agents do pay off: instant knowledge (APIs, legacy code) and cheap spikes that can be thrown away.

Source: U. Barbini, "Spec-Driven Development: The New Waterfall", Medium, August 2026.

The argument for intent: split the spec, assign owners

HUMAN OWNS

Intent

The outcome wanted, with no technology in it. Five parts: description, constraints, failure scenarios, success scenarios, links to other intents.

HUMAN OWNS

Expectations

The boundary: when the result counts as done, when it has failed, and the limits it must stay inside, in the user's terms.

HARNESS OWNS

Context

The how: tech stack, existing system, codebase constraints. Pulled in progressively by the tooling, not dumped into one document.

The agent builds and validates in a loop, but never decides what "done" means. People stay present in the loop instead of approving a large diff at the end.

Source: Ahuja's ICE frame (Intent, Context, Expectations), the basis of intent-driven software development (IDSD).

Two critiques of SDD, one direction

	Ahuja: intent-driven (IDSD)	Barbini: keep it agile
Diagnosis	One spec does four jobs; every gap becomes an agent decision	SDD repeats CASE and MDA; a full spec is as complex as the code
Replaces the big spec	Intent and expectations, written by the outcome owner	Executable, non-mocked tests; short docs checked against them
Frameworks	Harnesses, not methods	Scaffolding to learn from, not the destination
Way of working	Agent loops until expectations are met	Short loops, riskiest part first, replan as you learn
Human role	Owns intent and the definition of done, stays in the loop	Decides what the system should do and whether it is good enough

Combined: owners write intent and expectations; expectations become executable tests that stay the source of truth.

Sources: Ahuja (May 2026), Barbini (August 2026). The combined line is our synthesis, not either author's claim.

Three processes compared

Upfront design spec
Classic, document-driven

Full spec by humans → Sign-off → Humans build → Test at the end

Breaks: a "complete" spec is an illusion; gaps surface late, change is slow and costly.

SDD
Spec-driven, with agents

One spec: what + how → Plan and tasks → Agent builds, fills gaps → Human reviews diff

Breaks: same upfront illusion, but now the gaps are filled instantly and at volume. The diff is too large to truly review.

IDSD
Intent-driven, with agents

Intent + expectations → Harness pulls context → Code, validate, repeat → Merge when met

Changes: one outcome at a time, done is defined by its owner, the loop repeats until expectations are met. Needs people present throughout.

Classic critique: Larman and Basili (IEEE Computer, 2003); Ostroff et al. (XP 2004). IDSD flow: Ahuja (May 2026).

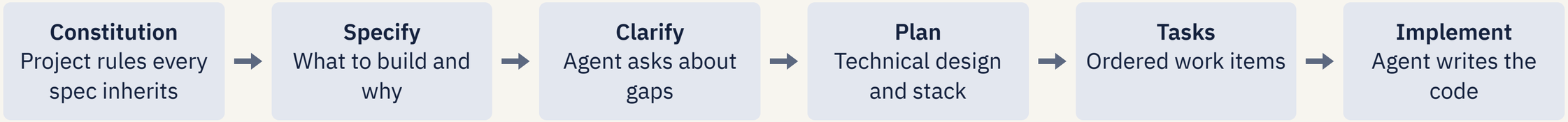
Current SDD frameworks: harnesses, not methods

Framework	Backer	Approach	Typical fit
Spec Kit	GitHub, open source	Spec-first CLI: specify, plan, tasks, implement	Greenfield; de facto default, 30+ agents
Kiro	AWS, commercial	Spec-native IDE: requirements, design, tasks	Guided workflow in one environment
OpenSpec	Fission AI, open source	Change proposals; specs persist and evolve	Brownfield, low ceremony
BMAD-METHOD	Community, open source	Multi-agent, role-based product documents	Large features, heavy process
Tessl	Tessl, commercial	Spec-as-source: edit spec, regenerate code	Strong traceability, higher lock-in
Superpowers	Community, open source	Skills for coding agents; task size sets rigor	Everyday work in one agent
GSD, PAUL	Community, open source	Guided plan-and-apply loops	Solo developers, small teams

Over 30 frameworks exist. All structure the spec-to-code workflow; none decides who owns intent and the definition of done. Use them as scaffolding to learn from, then shape your own process (Barbini).

As of October 2026. Sources: ssojet.com, softwareseni.com, codemyspec.com, github.com/cameronsjo/spec-compare, developertoolkit.ai.

Spec Kit in detail: GitHub's open-source SDD toolkit



What it is

- Released by GitHub in September 2025, MIT licence, free
- A CLI that installs templates and slash commands into the repository
- Works with 30+ coding agents; no lock-in to one vendor
- Most adopted SDD framework; best for greenfield work

Where it falls short

- Specs do not update during implementation, so they drift from the code
- Clarify and analyse steps are advisory; nothing gates the build
- No check that the running code does what the spec says
- Heavy ceremony for small changes and existing codebases

As of mid-2026. Sources: github.com/github/spec-kit, codemyspec.com, braingrid.ai, webreactiva.com. GitHub itself calls Spec Kit an experiment.

What a complete Spec Kit spec contains

spec.md: the chapters of the feature specification

Chapter	Content
1. User scenarios and testing (mandatory)	Prioritised user stories (P1 to P3), each with an independent test and Given/When/Then acceptance scenarios
2. Edge cases	Boundary conditions and error scenarios
3. Functional requirements (mandatory)	Numbered requirements (FR-001 …); unclear points flagged for clarification
4. Key entities (if data is involved)	Data objects, their meaning and relationships
5. Success criteria (mandatory)	Measurable, technology-agnostic outcomes (SC-001 …)
6. Assumptions	Defaults, scope boundaries, dependencies on existing systems

Documents around the spec

File	Content
constitution.md	Project-wide principles
plan.md	Summary, technical context, constitution check, project structure, complexity tracking
research.md	Technical decisions and options
data-model.md	Entities and fields
contracts/	Interface definitions
quickstart.md	How to run and validate
tasks.md	Ordered implementation tasks

Source: spec-template.md and plan-template.md in github.com/github/spec-kit (main branch, October 2026).

One feature, two ways: "Buy a red shoe for under $90"

Spec Kit: 8 documents up front

Open questions ("is burgundy red?", "is shipping inside $90?") are marked for clarification, then settled as technical decisions in research.md.

- spec.md · plan.md · research.md
- data-model.md · contracts · quickstart.md
- tasks.md (30 tasks) · constitution.md

IDSD: 2 owned files + context pointers

The product owner answers the same questions in the intent. Expectations give a reference shelf with "done when" and "failed if" checks.

- Intent INT-001 (human-owned)
- Expectations INT-001 (human-owned)
- Context sources (harness-owned)

Overview and side-by-side comparison: github.com/marcelRonner/sdd

Example product and stack are fictional; the IDSD file layout is one interpretation of the article.